30. Channels, Select, and Pipelines

30. Channels, Select, and Pipelines

Phase: Phase 5 — Concurrency
Estimated time: 2 days
Slug: `30-channels-select`

Learning objectives

- Create buffered and unbuffered channels
- Use select for multi-wait
- Build pipeline stages
- Close channels from the sender side only

Prerequisites

Complete prior numbered lessons. Type every example yourself.

Channels

```
ch := make(chan int)      // unbuffered
buf := make(chan int, 16) // buffered
ch <- 3
v := <-ch
close(ch)
```

Unbuffered send/receive synchronizes two goroutines. Buffer decouples up to capacity.

select

```
select {
case v := <-in:
    // handle
case <-ctx.Done():
    return ctx.Err()
case out <- v:
    // send
default:
    // non-blocking path
}
```

Pipelines

Stage functions: `in <-chan T` → `out <-chan U`, close out when done. Receivers use `for v := range ch`.

Common mistakes (especially from Python instincts)

- Closing channels from receivers
- Sending on closed channel (panic)

- Unbounded goroutine fan-out without backpressure

Exercises

Work in `exercises/30-channels-select/`. Read that folder's `README.md`.

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/30-channels-select/` until you have an honest failing attempt.

Self-check

1. Who should close a channel?
2. Buffered vs unbuffered?
3. What does range on channel do?

Further reading

- https://go.dev/blog/pipelines